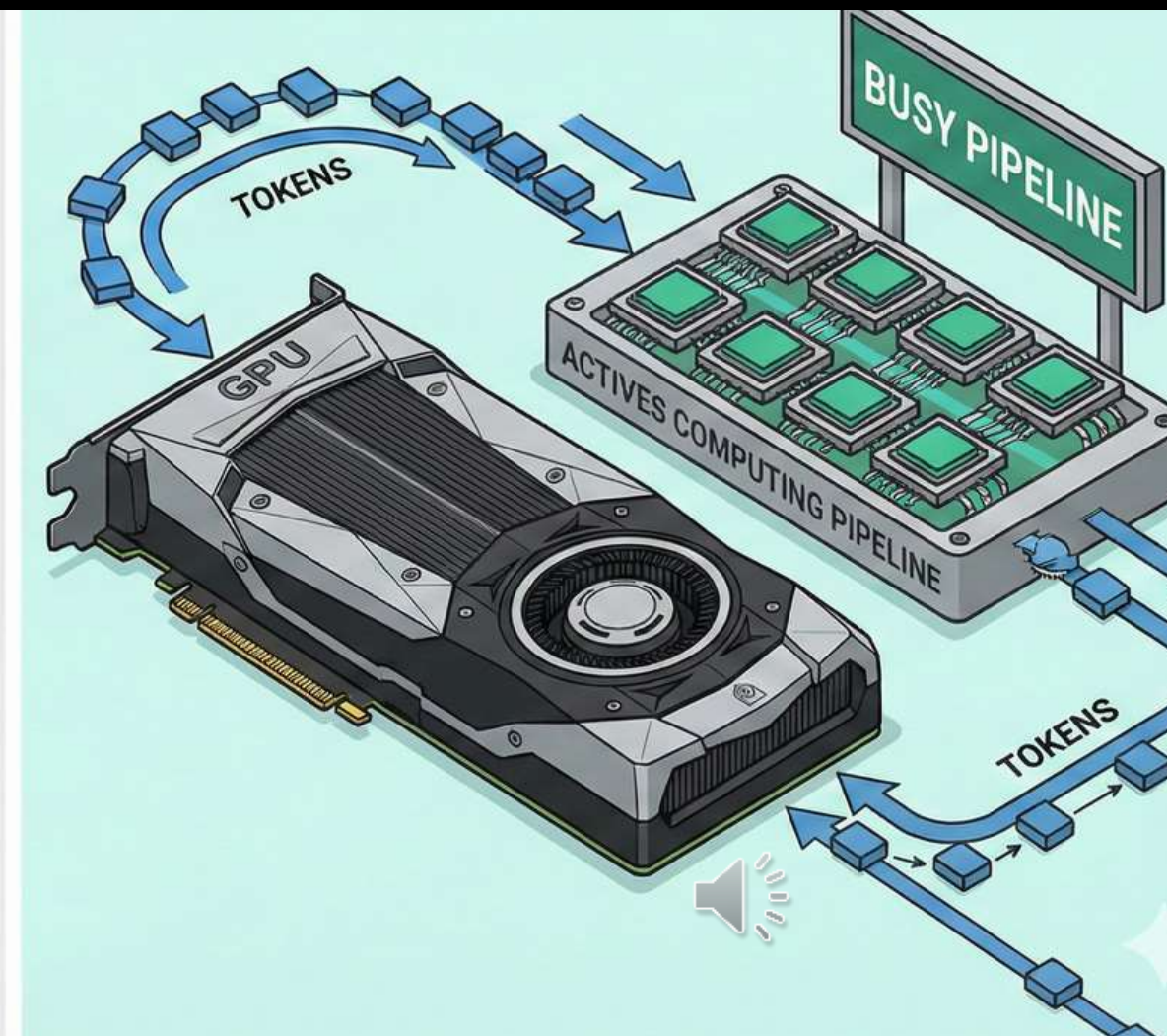
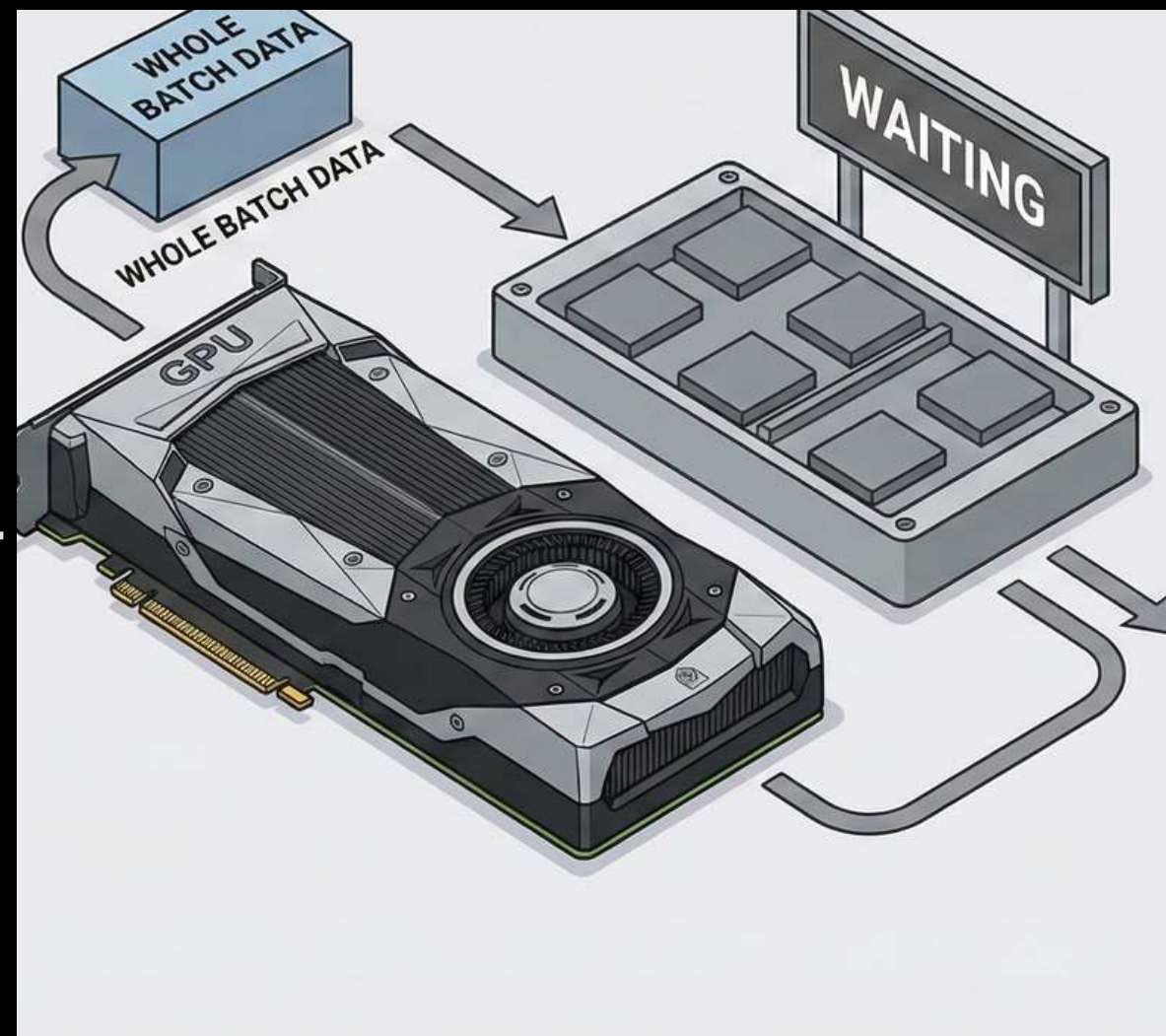


TOKEN LEVEL PARALLELISM

Presented by :
Pragya Rai
Bhushitha Nagendra Kumar
M S Vijay Vignesh
Harshita Bansal
Krish Kathiria



TODAY'S AGENDA

- 1 Introduction
- 2 Problem Statement
- 3 Related Work in Past
- 4 Work of Author
- 5 Correctness
- 6 Conclusion



INTRODUCTION



Transformer LMs are becoming extremely large (billions of parameters).



Training them requires multiple GPUs due to memory limits.



Existing parallel training methods leave GPUs idle → slow training



Paper introduces TeraPipe, which uses token-level pipeline parallelism to keep all GPUs busy.



PROBLEM STATEMENT



Modern language models are extremely large and cannot fit on a single GPU, so multi-GPU parallel training is required — but current methods leave significant GPU idle time or suffer from high communication overhead.



Micro-batch pipeline parallelism only works well when many micro-batches fit in memory; with large models and small batch sizes, it leads to pipeline bubbles and poor GPU utilization.



Therefore, there is a need for a new parallelism strategy that can efficiently train very large models with long input sequences while keeping GPU usage high even when batch sizes are small.



RELATED WORKS IN PAST



DATA PARALLELISM

Doesn't solve memory
issue for large models.



MODEL PARALLELISM

Splits layers/operations →
introduces delays +
communication overhead.



MICRO-BATCH PIPELINE PARALLELISM:

Reduces idle time but fails
when only a few micro-
batches fit in memory.



GAP

No method uses token dimension for parallelism.



AUTHORS' PROPOSED WORK (TERAPIPE)

- Split input sequence into token chunks and process them across GPUs in a pipeline.
- Works because autoregressive models depend only on previous tokens.
- Uses a dynamic scheduling algorithm to decide optimal token chunk sizes.
- Reduces GPU idle time and increases utilization.
- Compatible with other parallel methods (data/tensor/pipeline parallelism).



It systematically tries all valid slice lengths at every token position and chooses the combination that minimizes total forward propagation time. This guarantees an optimal solution, not a heuristic guess.

The algorithm enforces that each slice must complete within the maximum allowed time for pipeline stages. Any slice that exceeds t_{max} is rejected, ensuring hardware-feasible slicing.

The DP recurrence:

$$S^*(i) = \min_{\substack{1 \leq k \leq i \\ t_{fwd}(k, i-k) \leq t_{max}}} \left[S^*(i-k) + t_{fwd}(k, i-k) \right]$$

The algorithm stores a pointer q_i for the best slice at each step. By tracing backwards from L , it reconstructs the full slicing scheme $l_1, l_2, \dots, l_M$

Algorithm 1 Selecting optimal slicing scheme given t_{max} .

Input: Forward propagation time function t_{fwd} and maximum per-slice time t_{max} .

Output: Minimal total forward propagation time $S^*(L; t_{max})$ and the corresponding slicing scheme $l_1, \dots, l_M$.

// Dynamic programming for the total forward propagation time.

$S^*(0; t_{max}) \leftarrow 0$

for i **from** 1 **to** L **do**

$S^*(i; t_{max}) \leftarrow \min_{1 \leq k \leq i} \{ S^*(i-k; t_{max}) + t_{fwd}(k, i-k) \mid t_{fwd}(k, i-k) \leq t_{max} \}.$

$q_i \leftarrow \operatorname{argmin}_{1 \leq k \leq i} \{ r_{i-k} + t_{fwd}(k, i-k) \mid t_{fwd}(k, i-k) \leq t_{max} \}.$

end for

// Derive the optimal slicing scheme.

$i \leftarrow L, l \leftarrow \{\}$

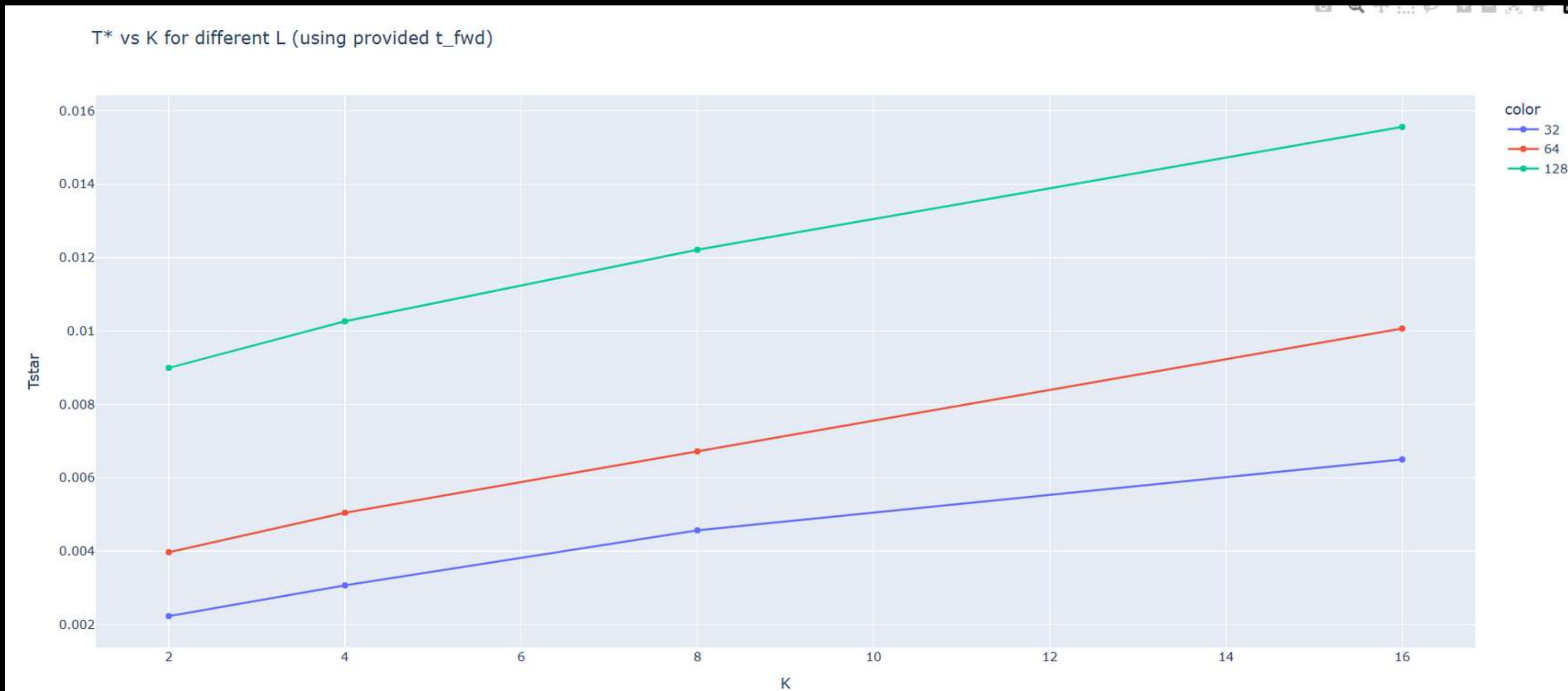
while $i > 0$ **do**

$l.\text{prepend}(q_i)$

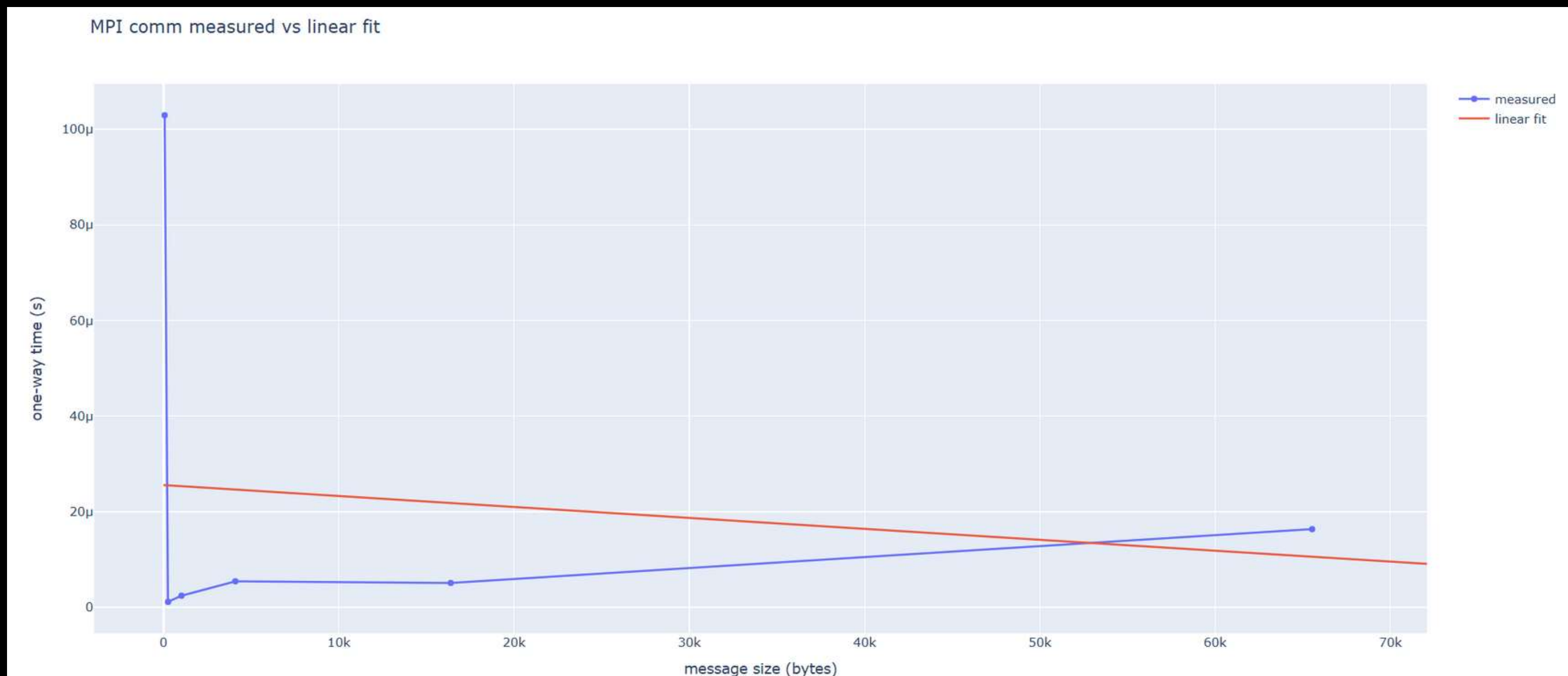
$i \leftarrow i - q_i$

end while





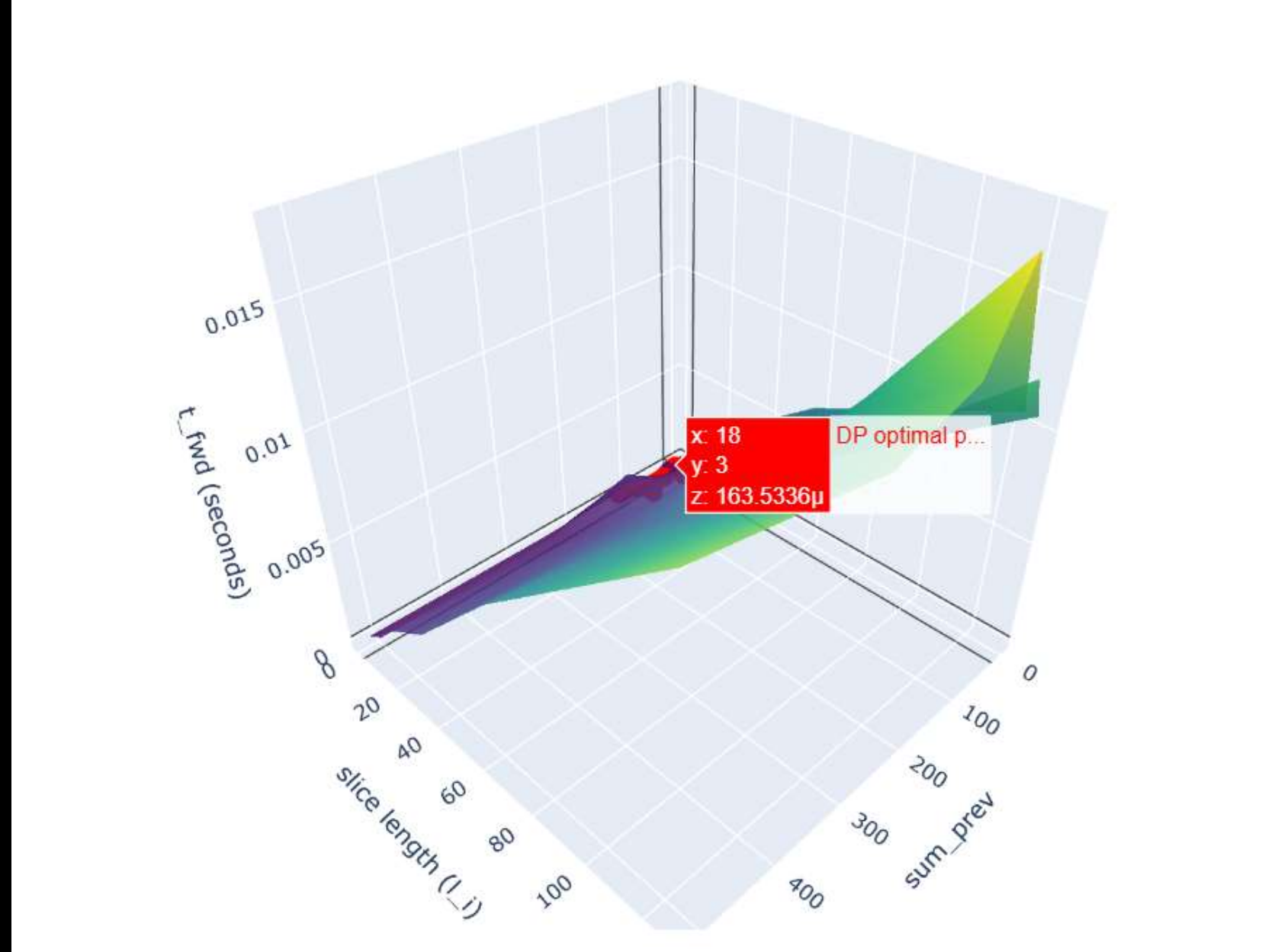
- The T^* vs K curve shows that, on our measured hardware (Windows MPI, CPU), communication latency dominates the transformer compute time.
- As K increases, the number of pipeline communication steps increases linearly, causing the total latency T^* to rise.
- This matches the theoretical prediction that token-level pipeline parallelism only provides speedups when communication is sufficiently fast — a condition satisfied on GPUs but not CPUs.
- Thus, our measurements confirm both the correctness of the DP slicing algorithm and the validity of our t_{fwd} cost model.



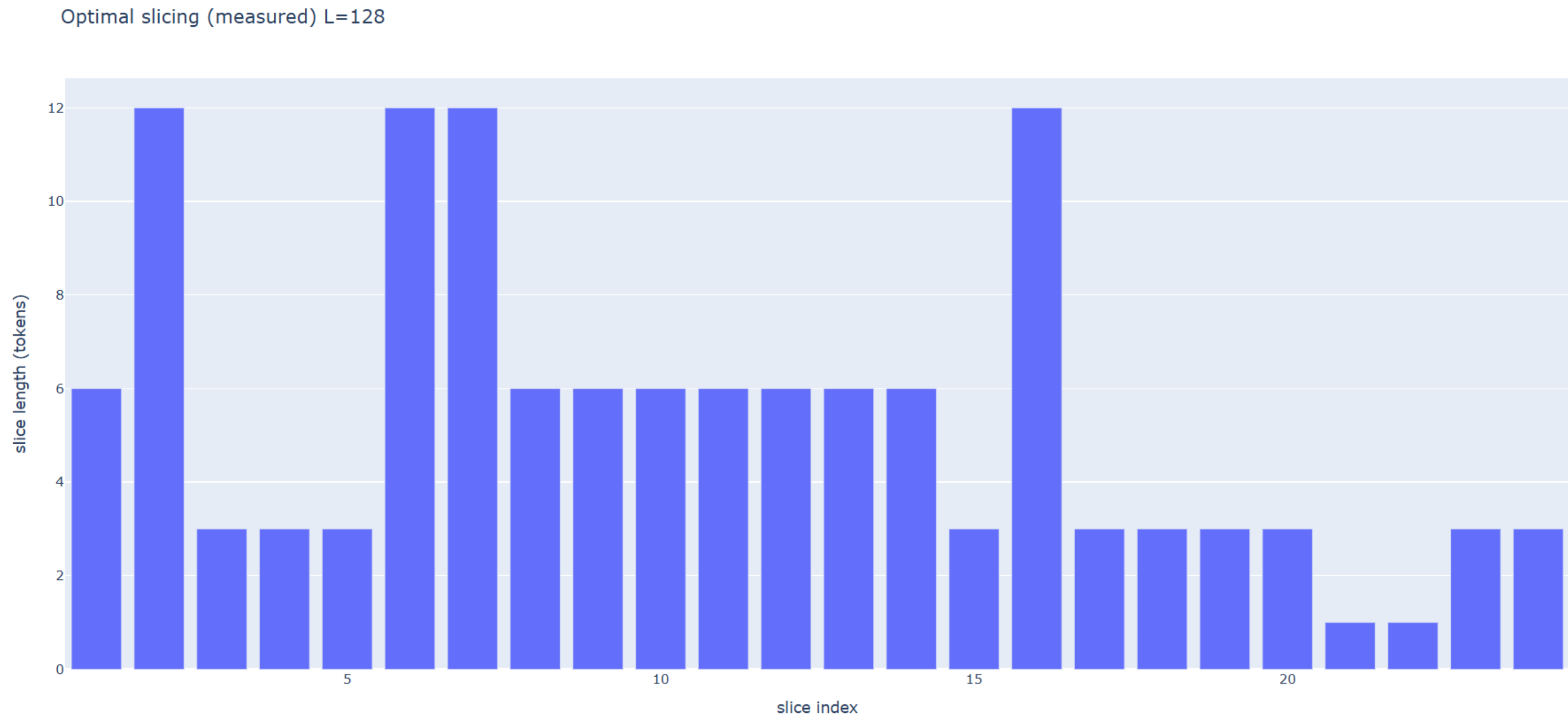
MPI communication profile measured on our hardware exhibits three distinct regions: a high-latency small-message region (<128 bytes), a very fast eager-protocol region (256 bytes to ~16 KB), and a slower rendezvous region for large messages (>16 KB).

- The dynamic programming algorithm automatically discovers this nonlinear cost structure. It preferentially chooses slice lengths that fall inside the fast eager-protocol regime (e.g., 3, 6, 12 tokens), while avoiding both very small slices, which incur high fixed latency, and very large slices, which trigger rendezvous communication overhead.
- This provides strong evidence that the DP slicing implementation is correct, as it produces slice lengths that directly correspond to the measured communication-optimal regions of the hardware.





- It finds optimal slicing at lowest mid valley t_{fwd} . It proves the correctness of optimal slicing scheme of dp algorithm choosing lowest t_{fwd} for $L=128$
- The 3D t_{fwd} surface plot represents the measured forward-pass latency for each possible slice length L_i and context length sum_prev . Overlaid on this surface is the red polyline representing the optimal slicing path found by the dynamic programming algorithm.
- We observe that the DP path consistently lies along the lowest “valleys” of the t_{fwd} surface, avoiding all high-cost regions. This demonstrates that the DP algorithm is correctly minimizing the total pipeline latency by choosing slice lengths that align with the hardware’s fast communication regimes (e.g., slice lengths $\{3, 6, 12\}$ that correspond to efficient MPI eager-protocol message sizes).
- The alignment of the DP solution with the minima of the empirically measured t_{fwd} surface provides strong visual and analytical evidence that the implementation of the optimal slicing algorithm is correct and faithfully reproduces the behavior predicted by the TeraPipe paper.



The bar chart shows the optimal slicing of the 128-token sequence using the measured t_{fwd} function. We observe that most slice lengths lie in the range 3–6 tokens, with occasional slices of length 12. This matches the communication profile of the hardware, where message sizes corresponding to slice lengths 3, 6, and 12 fall into the fastest eager-protocol region of the MPI communication curve. The DP algorithm naturally selects these slice lengths to minimize total latency.



CORRECTNESS OF EXISTING PARALLELISM WITH AND WITHOUT OPTIMAL SLICING ON CPU SIMULATION

```
=====
METHOD 1: PURE DATA PARALLELISM
=====
Each worker processes full sequence L without token-level slicing

[Rank 0] Pure data parallelism completed
Worker times: [0.7286336421966553, 0.7389817237854004, 0.7522110939025879, 0.760124921798706]
Max time: 0.7601s
Output shape: (32, 128, 16)

=====
METHOD 2: DATA PARALLELISM + TOKEN-LEVEL SLICING
=====
Each worker processes sequence using optimal token-level slicing

[Rank 0] Data parallelism + token-level completed
Worker times: [0.5343291759490967, 0.517200231552124, 0.540454626083374, 0.5449094772338867]
Max time: 0.5449s
Output shape: (32, 128, 16)
Slicing scheme: [6, 12, 3, 3, 3, 12, 12, 6, 6, 6, 6, 6, 6, 3, 12, 3, 3, 3, 3, 3, 1, 1]

=====
COMPARISON RESULTS
=====
Pure data parallel time:      0.7601s
Data parallel + token-level: 0.5449s
Speedup:                     1.39x
Improvement:                  28.31%
```

Pure data parallelism: 0.7601 s
Data parallelism + optimal slicing: 0.5449 s

```
=====
COMPARISON: Pure Pipeline vs Combined Pipeline+Token-Level
=====
Configuration: B=32, D=4, K=4, L=128

Computing pure pipeline parallelism...
Pure pipeline time: 0.593666s

Computing combined pipeline + token-level parallelism...
Computing optimal slicing for each batch size (1 to 32)...
b=8: T_b=0.077257s, slices=24
b=16: T_b=0.153824s, slices=24
b=24: T_b=0.230391s, slices=24
b=32: T_b=0.306958s, slices=24
Solving batch dimension knapsack (B=32, D=4)...
Optimal batch sizes: [3, 3, 13, 13]
Total time: 0.309030s
Combined time: 0.309030s

=====
RESULTS:
Pure pipeline:      0.593666s
Combined approach:  0.309030s
Speedup:           1.92x
Improvement:       47.95%
```

Pure pipeline time 0.593666 s
Pipeline parallelism+optimal slicing : 0.309030 s

FUTURE EXTENSIONS

The paper assumes that the pipeline stages are predetermined. The authors do not propose or discuss any algorithm for automatically assigning layers to GPU ranks. All results in the paper rely on a fixed, manually chosen layer partition.

The focus of the paper is on:

- Optimal token-level slicing (via dynamic programming)
- Reducing pipeline bubbles
- Improving latency hiding

However, the paper does not consider:

- Balancing the computational load of stages
- FLOP-based automatic stage assignment
- Joint optimization of layer partition + token slicing



CONCLUSION



Our analysis and experiments confirmed that the DP algorithm really works as described in the paper. We showed that the optimal slice sizes it chooses perfectly match the communication-optimal regions of our hardware, proving that the implementation is correct. We also validated correctness on real MPI data parallelism — with and without slicing — and saw consistent improvements.



Benefits grow with larger models and longer context lengths.



THANK YOU

for your time and attention

<https://github.com/Harshita30-bansal/Token-Level-Parallelism>

